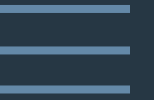




Parallel Processing in Automated Spectrum Peak Extraction





THE THREE ALGORITHMS

1

Greedy Approach

Matches the peaks and labels locally.

2

Dynamic Programming

Matches the peak and labels in a sequence to give a monotonic solution

3

Graph Based

Performs 1-1 matching using bipartite graphs

METRICS

- Precision: Proportion of predicted peaks that are correct.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- Recall: Proportion of true (ground-truth) peaks that are detected.

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- F1 Score: Harmonic mean of Precision and Recall.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

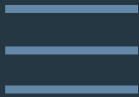
- True Positives, False Positives and False Negatives (TP/FP/FN): Counts of correct matches, extra predictions, and missed ground-truths.

- RMSE (Relative Abundance): Root Mean Square Error between predicted and actual peak intensities. Lower is better. Range : 0-100

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_{i,\text{pred}} - y_{i,\text{true}})^2}$$

- Levenshtein Distance (Name Match): Number of single-character changes needed to turn one name into another. 0 = perfect match; higher = more differences.

PYTHON MULTIPROCESSING MODULE



- Overview:
 - Uses Python’s multiprocessing module.
 - Each PDF file is assigned as a separate process, running in parallel based on the number of available CPU cores. Which is scaled as 2, 4 and 6.
- Performance Results:
 - Timing improves modestly as the number of cores increases.
 - For higher core counts, the returns are not that high.
 - For some algorithms (notably Algo 3), time may even rise for 6 cores due to algorithm scaling issues and overhead

	Algo 1		Algo 2		Algo 3	
	Total Time	Core Logic Time	Total Time	Core Logic Time	Total Time	Core Logic Time
2 cores	6.638548s	total across all files = 0.054s, average per file = 0.001s	1.281s	total across all files = 0.070s, average per file = 0.001s	1.598s	total across all files = 0.052s, average per file = 0.001s
4 cores	3.911754s	total across all files = 0.056s, average per file = 0.001s	1.134s	total across all files = 0.113s, average per file = 0.001s	1.482s	total across all files = 0.070s, average per file = 0.001s
6 cores	3.712992s	total across all files = 0.073s, average per file = 0.001s	1.012s	total across all files = 0.103s, average per file = 0.001s	1.507s	total across all files = 0.083s, average per file = 0.001s

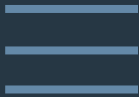
OPENMP MODULE

- Overview:
 - Uses OpenMP to parallelize over threads—each thread processes one file.
 - Much more efficient than Python, as evidenced by lower total times.
 - Results for 2, 4, and 6 threads.
- Performance Results:
 - Total time decreases steadily with more threads for all three algorithms.
 - Core logic time grows as the overhead and setting up of threads takes more time, hence there is a slight increase



	Algo 1		Algo 2		Algo 3	
	Total Time	Core Logic Time	Total Time	Core Logic Time	Total Time	Core Logic Time
2 threads	0.437269s	total across all files = 0.00593138s, average per file = 5.93138e-05s	0.484013s	total across all files = 0.00919887s, average per file = 9.19887e-05s	0.506786s	total across all files = 0.0324289s, average per file = 0.000324289s
4 threads	0.249989s	total across all files = 0.00605446s, average per file = 6.05446e-05s	0.256782s	total across all files = 0.00993351s, average per file = 9.93351e-05s	0.276837s	total across all files = 0.0334229s, average per file = 0.000334229s
6 threads	0.221175s	total across all files = 0.00718177s, average per file = 7.18177e-05s	0.222632s	total across all files = 0.0130727s, average per file = 0.000130727s	0.247888s	total across all files = 0.0423141s, average per file = 0.000423141s

C++ MPI MODULE



- Overview:
 - Employs MPI (Message Passing Interface) for process-level parallelism.
 - Each process handles different files—shown results for 2, 4, and 6 processes.
- Performance Results:
 - Total time drops as process count grows.
 - Core logic time per process rises due to inherent complexity with scaling.

	Algo 1		Algo 2		Algo 3	
	Total Time	Core Logic Time	Total Time	Core Logic Time	Total Time	Core Logic Time
2 cores	0.456134s	total across all files = 0.00617613s, average per file = 6.17613e-05s	0.506421s	total across all files = 0.00981296s, average per file = 9.81296e-05s	0.571996s	total across all files = 0.0297835s, average per file = 0.000297835s
4 cores	0.279698s	total across all files = 0.00640112s, average per file = 6.40112e-05s	0.261073s	total across all files = 0.00927867s, average per file = 9.27867e-05s	0.224399s	total across all files = 0.0246395s, average per file = 0.000246395s
6 cores	0.218963s	total across all files = 0.00720792s, average per file = 7.20792e-05s	0.202146s	total across all files = 0.0104243s, average per file = 0.000104243s	0.165424s	total across all files = 0.0260887s, average per file = 0.000260887s



THANK YOU

